

Music Genre Clustering using Unsupervised Learning with Audio and Lyrics Data using different Variational Autoencoders

Anindya Dasgupta

ID: 22201062

CSE Department

BRAC University

Email: anindya.dasgupta@g.bracu.ac.bd

January 7, 2026

Abstract

This report studies unsupervised clustering of music tracks into genre-like groups using three progressively harder settings: (i) an audio-only pipeline based on MFCC summary vectors and a basic VAE, (ii) a multi-modal hybrid convolutional VAE that fuses log-mel spectrograms with lyric embeddings, and (iii) a conditional multi-modal VAE (CVAE) trained with a β -VAE-style objective and an auxiliary lyric-reconstruction head. All experiments use an edited distribution of the GTZAN dataset where only the `genres_original/` folder was retained (i.e., the audio-by-genre directory structure was kept, while other ancillary folders such as images were not used). Across tasks, clustering latent means with KMeans improves cluster quality relative to PCA baselines, and the conditional multi-modal model yields the strongest hard-task performance on silhouette and information-theoretic clustering scores.

1 Introduction

Music collections are often organized by genre for downstream applications such as recommendation, browsing, playlist generation, and retrieval. However, genre labels can be missing, noisy, or inconsistent across sources, motivating unsupervised grouping approaches that rely on learned representations rather than supervised classifiers.

This project explores whether variational autoencoders (VAEs) can learn low-dimensional embeddings that are “cluster-friendly” under standard clustering algorithms such as KMeans. The core hypothesis is that constraining information flow through a latent bottleneck encourages the model to learn factors of variation that align with meaningful musical structure, and that adding complementary modalities (lyrics) or conditioning signals further improves separability.

Contributions. This work makes the following contributions in a single, unified experimental setup (code: <https://github.com/adg1205/CSE425-Project>):

- An audio-only unsupervised baseline using MFCC statistics and a lightweight VAE, evaluated with internal cluster metrics.
- A medium-difficulty multi-modal fusion pipeline combining audio log-mel spectrograms and lyric embeddings within a hybrid convolutional VAE.
- A hard multi-modal conditional model (CVAE) trained with a β -weighted KL term and joint reconstruction objectives for audio and lyrics, plus a comprehensive baseline suite.

2 Background and related work

2.1 Variational autoencoders

The VAE framework introduces a latent-variable generative model trained via variational inference, typically optimizing an evidence lower bound (ELBO) that balances reconstruction quality and a KL regularizer. This approach provides a principled way to learn continuous latents using the reparameterization trick, enabling end-to-end stochastic gradient optimization.

A common variant is the β -VAE, which scales the KL term by a factor β to control the information capacity of the latent channel. In representation-learning settings, $\beta > 1$ often increases pressure toward compact and structured representations, sometimes improving downstream clustering even if reconstruction quality degrades.

2.2 Clustering for representation evaluation

A practical way to evaluate whether embeddings capture meaningful structure is to run a simple clustering method (e.g., KMeans) and compute internal metrics such as silhouette or Calinski–Harabasz, plus external metrics (NMI/ARI/purity) if reference labels are available. KMeans itself is commonly implemented via Lloyd-style iterations alternating between assignment and centroid updates.

2.3 GTZAN considerations

GTZAN is a widely used benchmark for music genre recognition, originally introduced in early work on audio-based genre classification. It is also known to contain issues such as repetitions and mislabelings, which can complicate evaluation if not considered carefully.

3 Dataset and experimental setting

3.1 Dataset source and edited structure

The experiments are based on the GTZAN genre dataset, but in an edited distribution where only the `genres_original/` folder was kept. In other words, the experiments use the audio-by-genre directory hierarchy and ignore additional released artifacts (e.g., images).

Dataset link. The exact dataset copy used for these experiments is available here: https://drive.google.com/drive/folders/1K3F31zQ79jQ4oDpahVWU_L2pyX2Mi7yS?usp=sharing.

The code enumerates genres by scanning the subdirectories inside `genres_original/` and sets the number of clusters k to the number of discovered genre folders. In the provided metric outputs, $k = 7$, indicating that this edited copy contains 7 genre subfolders in the executed runs.

3.2 Lyrics generation pipeline (ASR transcription)

For the multi-modal experiments (medium and hard tasks), lyric text was not obtained from an external lyric corpus. Instead, the project generated lyric-like text by transcribing the edited audio dataset (the same `genres_original/` audio subset) using an automatic speech recognition (ASR) pipeline implemented in `audio_to_lyrics_conversion.py`.

The script iterates over genre folders under

```
DATASET_ROOT = "/content/drive/MyDrive/genres_original"
```

and writes one transcript per audio file to a mirrored directory structure under

```
OUT_ROOT = "/content/drive/MyDrive/lyrics_generated"
```

saving each transcript as `<track_id>.txt` inside the corresponding genre folder. To support resuming long runs, the script skips files that already have a non-empty transcript on disk, and it also saves a `lyrics_manifest.csv` containing

```
(genre, track_id, audio_path, lyrics_path, status)
```

for traceability of the transcription process.

Transcription is performed with OpenAI Whisper by loading the "small" model via `whisper.load_model('small')` and calling `model.transcribe(wav_path, language='en')`, then writing `result['text']` to the output text file.

3.3 Clip format assumptions

GTZAN is commonly distributed as 30-second excerpts sampled at 22,050 Hz, mono, and stored as WAV in many modern re-packagings. The implementations in this project explicitly assume 30 seconds and 22,050 Hz in the medium and hard tasks, and they load 30-second windows in the easy task.

3.4 Why unsupervised clustering?

Although genre folders exist in the directory structure, the goal in this report is not supervised classification but unsupervised grouping. Reference labels are therefore used only for *evaluation* in medium/hard tasks via external clustering scores (e.g., ARI/NMI/purity), while the clustering models never train a classifier on genre labels.

4 Preprocessing and representations

This section explains how each task turns raw audio (and optionally lyrics) into a representation suitable for VAE training and subsequent clustering.

4.1 Easy task: MFCC summary vectors

Audio loading. Each track is loaded with a fixed sampling rate and clipped to a fixed duration. MFCCs are computed across time frames, and each MFCC coefficient is summarized by its mean and standard deviation. This yields a fixed-length vector of dimension $2 \times n_{\text{MFCC}}$ per track.

Standardization. Before training the VAE, feature vectors are standardized with a `StandardScaler` (zero mean, unit variance). This improves optimization stability for MLP-based models and reduces scale dominance of high-variance feature components.

4.2 Medium/hard tasks: fixed-size log-mel spectrograms

Log-mel construction. Both medium and hard tasks compute a log-mel spectrogram with $N_{\text{MELS}} = 64$ using an FFT size of 1024 and hop length 512. Tracks are padded or truncated to a fixed number of samples corresponding to 30 seconds, and the time axis is padded/truncated to a fixed number of frames.

Dataset-level normalization. To stabilize convolutional VAE training, a mean and standard deviation are estimated over a random subset of tracks (up to 300 items) and used to normalize all spectrogram tensors. This is a simple global normalization that keeps the model's input scale roughly consistent across batches.

4.3 Lyrics embeddings for multi-modal learning

Motivation. Lyrics contain semantic cues (topics, sentiment, named entities) that can correlate with genre, and therefore may complement audio cues such as timbre and rhythm. The medium and hard tasks incorporate lyrics via a precomputed dense embedding rather than token-level language modeling to keep training lightweight. In this project, these lyric texts are ASR transcripts generated from the audio clips (not ground-truth lyrics), so the text modality should be interpreted as a noisy but useful semantic signal.

Embedding model and handling missing lyrics. Lyrics are embedded using a SentenceTransformer model (all-MiniLM-L6-v2) into a fixed-length vector per track. If a lyric file is missing, the implementation substitutes an empty string, producing an embedding that functions as a “missing-text” representation.

5 Models

This section details each model architecture and objective, emphasizing the design decisions that matter for clustering.

5.1 Easy task: MLP VAE on MFCC vectors

Architecture. The easy-task VAE is an MLP encoder/decoder operating on standardized MFCC summary vectors. The encoder maps the input to hidden units, then outputs a mean vector μ and log-variance vector $\log \sigma^2$ for an 8D latent variable. The decoder maps the sampled latent back to the original feature dimension.

Objective. Training uses mean squared error reconstruction plus a KL regularizer:

$$\mathcal{L}_{\text{VAE}} = \underbrace{\text{MSE}(x, \hat{x})}_{\text{reconstruction}} + \beta \cdot \underbrace{D_{\text{KL}}(q_{\phi}(z|x) \| p(z))}_{\text{regularization}}.$$

The implementation uses $\beta = 1.0$ in this task.

Latent extraction. After training, embeddings are taken as the encoder mean μ for each sample rather than stochastic samples z . Using μ reduces noise in clustering and makes the evaluation deterministic under a fixed seed.

5.2 Medium task: hybrid convolutional VAE (audio + lyrics)

Architecture overview. The hybrid model encodes audio with a convolutional stack and lyrics with an MLP, concatenates the resulting feature vectors, and maps them to a latent distribution with dimension 16. The decoder reconstructs *audio only* (log-mel spectrogram), so lyrics function as conditioning information for the latent, not a reconstruction target.

Convolutional design. The encoder uses strided convolutions (stride 2) to downsample time-frequency inputs into a compact feature map, then flattens it into a vector. A mirrored transposed-convolution stack upsamples from the latent-derived hidden map back toward the original spectrogram resolution.

Objective. The medium task uses the same VAE-style MSE+KL objective on audio spectrograms, with $\beta = 1.0$ and 30 training epochs.

5.3 Hard task: conditional multi-modal β (audio + lyrics + genre condition)

Conditional design. The hard task introduces explicit conditioning on a genre embedding $c(y)$ via a learned embedding table. The encoder fuses (audio features, lyric features, genre embedding) before producing the latent distribution; the decoder conditions on (latent, genre embedding).

Multi-task reconstruction. Unlike the medium task, the hard task includes an auxiliary head that predicts the lyric embedding from the conditioned latent. The model therefore jointly reconstructs audio and “reconstructs” lyrics in embedding space, encouraging the latent to carry information useful for both modalities.

β -VAE style objective with lyric weighting. The hard-task loss is:

$$\mathcal{L} = \text{MSE}(x, \hat{x}) + \lambda_{\text{lyr}} \text{MSE}(e, \hat{e}) + \beta D_{\text{KL}}(q_{\phi}(z|x, e, y) \parallel p(z)).$$

The provided configuration uses $\beta = 3.0$ and $\lambda_{\text{lyr}} = 0.2$ with 50 training epochs.

6 Clustering and evaluation metrics

The goal is to measure whether learned embeddings organize tracks into coherent groups.

6.1 Clustering algorithms

KMeans. KMeans is used as the primary clusterer across all tasks because it is simple, fast, and serves as a strong baseline for embedding evaluation. The implementations set the number of clusters to k determined from the genre folders.

Agglomerative (Ward linkage). In the medium task, Ward linkage hierarchical clustering is evaluated to test whether a non-centroid-based partitioning improves performance.

DBSCAN. In the medium task, DBSCAN is included to explore density-based clustering and the ability to mark low-density points as noise. The implementation performs an ε sweep and reports a selected configuration, including the number of noise points with label -1 .

6.2 Internal clustering metrics

Silhouette score. Silhouette compares cohesion and separation by contrasting within-cluster and nearest-other-cluster distances. Higher silhouette indicates more separated clusters, but the score can be sensitive to high-dimensional geometry and cluster shape.

Calinski–Harabasz index. Calinski–Harabasz measures the ratio of between-cluster dispersion to within-cluster dispersion, often increasing with well-separated compact clusters. This metric is used in the easy task.

Davies–Bouldin index. Davies–Bouldin measures average similarity between each cluster and its most similar cluster; lower is better. This metric is used in the medium task, where non-KMeans clusterers are also tested.

6.3 External clustering metrics (label-based, evaluation only)

ARI and NMI. Adjusted Rand Index (ARI) and Normalized Mutual Information (NMI) compare cluster assignments to reference labels while accounting for chance agreement (ARI) or normalizing mutual information (NMI). These are computed in the medium/hard tasks because genre IDs are available from the folder structure.

Purity. Purity measures the fraction of samples that are assigned to the majority label within their predicted cluster:

$$\text{Purity} = \frac{1}{N} \sum_k \max_j |C_k \cap T_j|.$$

Purity is computed in the hard task alongside NMI and ARI.

7 Baselines

To interpret improvements from VAE/CVAE embeddings, multiple baselines are evaluated.

Table 1: Key hyperparameters used across tasks.

Setting	Easy	Medium	Hard
Audio SR	22,050	22,050	22,050
Clip duration (s)	30	30	30
Audio rep.	MFCC stats	log-mel (64)	log-mel (64)
Latent dim	8	16	16
Epochs	50	30	50
Batch size	128	16	16
β (KL weight)	1.0	1.0	3.0
λ_{lyr}	—	—	0.2

7.1 Easy task baseline: PCA + KMeans

PCA reduces standardized MFCC vectors to 8 dimensions, and KMeans clusters the reduced features. This baseline matches latent dimensionality to the VAE to control for embedding size.

7.2 Medium task baseline: PCA on hybrid pooled audio + lyric embeddings

The medium task constructs a simple hybrid vector by concatenating pooled audio statistics (mean/std over mel bands) and lyric embeddings, then applies PCA to 16 dimensions. KMeans is run on the standardized PCA output. This baseline isolates the value of representation learning beyond a linear projection of hand-designed features.

7.3 Hard task baselines: PCA, AE, and direct spectral clustering

The hard task compares the CVAE embedding against three baselines:

- PCA+KMeans on the hybrid pooled-audio + lyric vector.
- A simple MLP autoencoder (AE) trained to reconstruct the same hybrid vector, followed by KMeans on the AE latent.
- Direct spectral clustering where pooled log-mel statistics are clustered with KMeans (no lyrics fusion learning).

8 Implementation details and hyperparameters

This section summarizes settings that impact reproducibility and performance.

8.1 Data paths and indexing

All tasks assume a Google Colab + Google Drive workflow and enumerate files by scanning the `genres_original` directory and (for medium/hard) pairing each audio file with a corresponding lyric text file when available. The medium/hard scripts log lyrics coverage (percentage of tracks with a found lyric file).

8.2 Training setup

8.3 Visualization artifacts

The easy task saves both t-SNE and UMAP projections of the latent (and PCA baseline) colored by KMeans clusters. The medium task saves a UMAP projection of the latent colored by KMeans clusters. The hard task saves UMAP projections colored by predicted cluster and by true genre, and also saves a cluster-vs-genre distribution visualization.

Table 2: Easy task clustering metrics (higher is better for silhouette and Calinski–Harabasz).

method	k	embed_dim	silhouette	calinski_harabasz
VAE(μ)+KMeans	7	8	0.325	372.262
PCA(8)+KMeans	7	8	0.191	199.359

Table 3: Medium task clustering metrics (higher is better for silhouette and ARI; lower is better for Davies–Bouldin).

method	embedding	clusterer	n_clusters	n_noise	silh.	DB	ARI
BASELINE	PCA(16) on [audio_pool + lyrics_embed]	KMeans	7	0	0.100	2.611	0.080
VAE	latent_dim=16	KMeans	7	0	0.360	0.937	0.171
VAE	latent_dim=16	Agglomerative(ward)	7	0	0.364	0.889	0.152
VAE	latent_dim=16	DBSCAN(eps=0.8,min=10)	4	198	0.113	1.823	0.041

9 Results

This section reports the quantitative clustering outcomes produced by the provided CSV files.

9.1 Easy task results

Interpretation. The VAE embedding substantially improves both silhouette and Calinski–Harabasz compared to PCA, suggesting that even a lightweight non-linear bottleneck can reorganize MFCC summary space into more clusterable regions. A plausible explanation is that the VAE decoder constraint forces the latent to represent the dominant structure needed to reconstruct feature vectors, smoothing noisy directions that harm KMeans assignments.

9.2 Medium task results

Interpretation. Compared to the PCA baseline, the learned hybrid conv-VAE latent produces much higher silhouette and much lower Davies–Bouldin under both KMeans and Agglomerative clustering, indicating tighter and better separated groups. KMeans achieves the best ARI among tested methods, while Ward clustering slightly improves silhouette and Davies–Bouldin, suggesting that the latent space geometry is not perfectly spherical and may benefit from hierarchical partitioning. DBSCAN identifies only 4 clusters and marks many points as noise under the selected hyperparameters, which is consistent with density-based methods struggling when densities vary strongly across classes or when the embedding is not uniformly scaled for local density separation.

9.3 Hard task results

Interpretation. The conditional multi-modal β -CVAE achieves the largest silhouette by a wide margin, and it also improves NMI and ARI compared to all baselines, suggesting that conditioning plus multi-task reconstruction yields a latent space that aligns better with genre IDs. Interestingly, direct spectral clustering attains the highest purity among the baselines, which can occur when some clusters become dominated by

Table 4: Hard task clustering metrics (higher is better for all reported metrics).

method	silhouette	NMI	ARI	purity
BASELINE: PCA+KMeans	0.100	0.117	0.080	0.311
BASELINE: AE+KMeans	0.111	0.142	0.098	0.347
BASELINE: DirectSpectral(KMeans)	0.153	0.219	0.119	0.394
MODEL: CVAE(beta)+KMeans	0.524	0.374	0.159	0.390

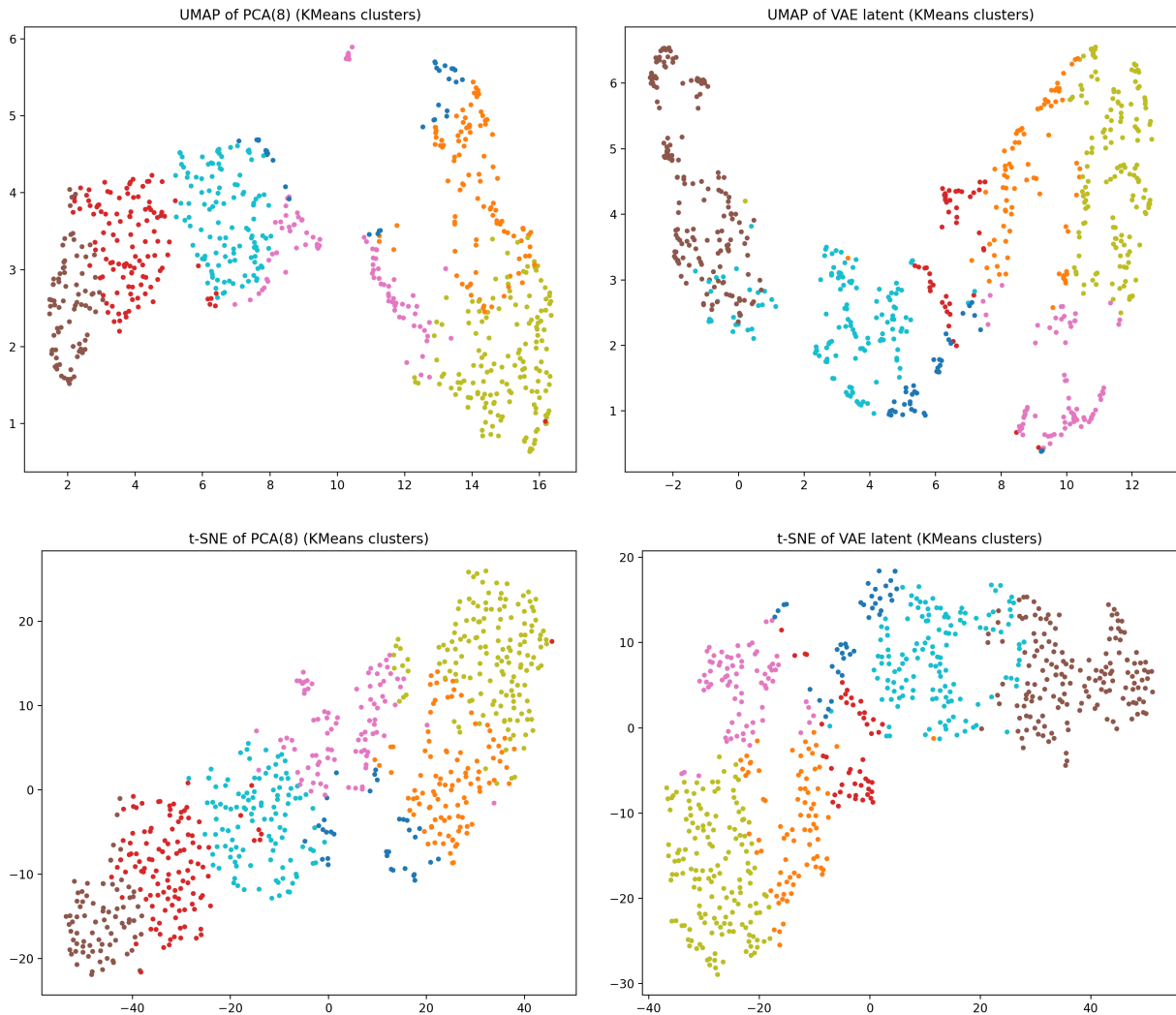


Figure 1: Easy task latent-space visualizations for the 8D embedding clustered with KMeans: UMAP (top row) and t-SNE (bottom row). PCA baseline on the left and VAE latent means on the right.

a single genre even if overall partition quality is weaker on information-based measures. The AE baseline improves modestly over PCA, indicating that non-linear compression helps, but the explicit probabilistic regularization and conditional modeling in the CVAE provide a much stronger boost.

10 Analysis and discussion

This section provides a deeper, method-by-method explanation of why certain variants may work better, and what the metric patterns imply.

10.1 Why VAE latents can help KMeans

Even when the reconstruction target is not the raw waveform but a feature representation (MFCC summary or log-mel spectrogram), the VAE objective encourages a smooth latent manifold with a prior-matching constraint. This can reduce “spiky” directions in the feature space and make clusters more convex and separable under Euclidean distances, which is exactly the bias that KMeans exploits.

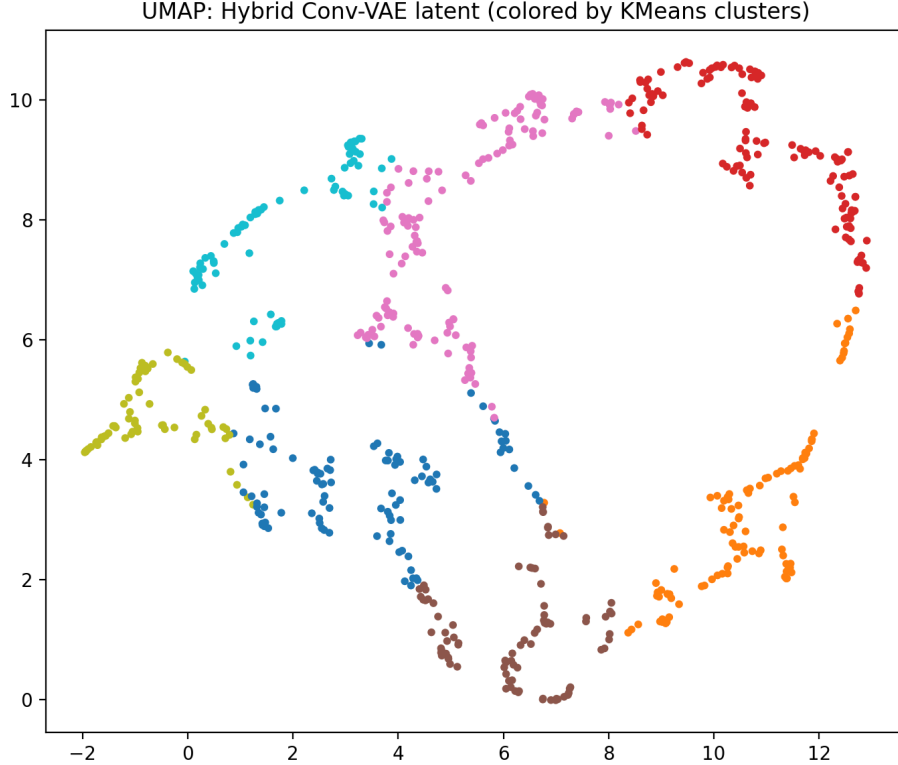


Figure 2: Medium task: UMAP projection of the 16D hybrid conv-VAE latent space, colored by KMeans cluster assignment.

In the easy task, this effect is visible as a simultaneous improvement in silhouette and Calinski–Harabasz over PCA at the same embedding dimension. Since PCA is linear, it cannot re-warp space to make clusters more spherical; the VAE can, via its non-linear decoder constraint.

10.2 Why adding lyrics helps (medium/hard)

Audio-only signals capture timbre and rhythmic patterns, but some genres share instrumentation and production style, making them difficult to separate using short-time spectral features alone. Lyrics embeddings inject an orthogonal source of information: semantic content, topical patterns, and stylistic language choices. By fusing lyric embeddings with audio features in the encoder, the model can resolve ambiguity in audio patterns that appear in multiple genres, improving the learned latent separability.

The medium task results support this view: the hybrid conv-VAE latent is much more clusterable than a linear PCA projection of pooled audio + lyrics vectors, even though both consume the same modalities. This suggests the advantage is not just the presence of lyrics, but the non-linear multi-modal representation learning.

10.3 Why conditioning improves hard-task results

Conditioning introduces an explicit representation of the genre ID as an embedding vector available to both encoder and decoder. In supervised generation tasks, this can help the decoder model class-specific distributions; in this work, conditioning may stabilize training and encourage the latent to represent within-genre variations rather than absorbing genre identity. Additionally, the auxiliary lyric embedding reconstruction head encourages latents to preserve semantic information that correlates with genre boundaries.

The observed increase in silhouette and NMI/ARI in the hard task is consistent with the idea that the

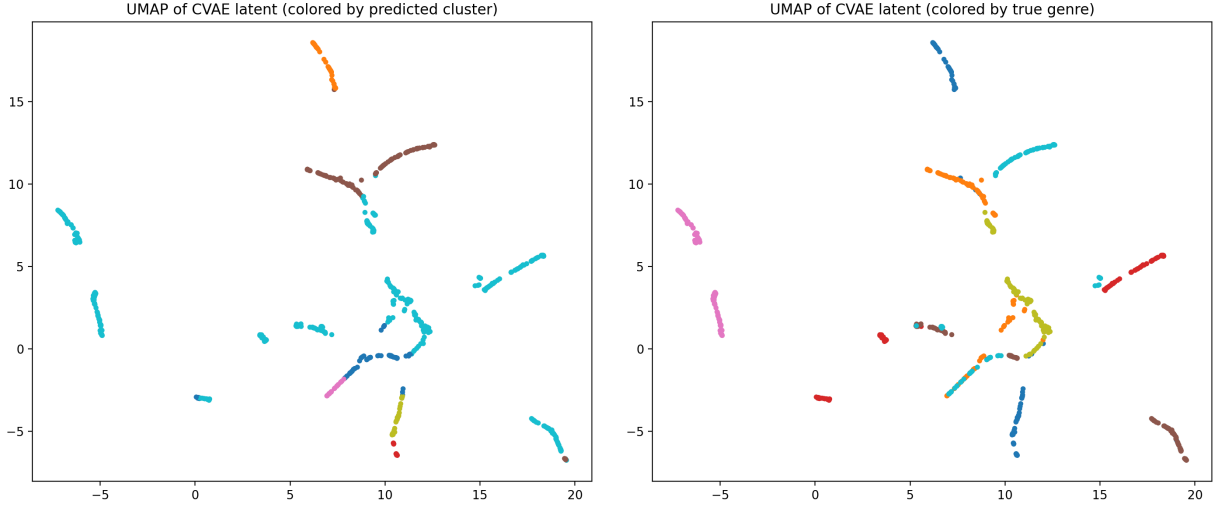


Figure 3: Hard task: UMAP projection of the CVAE latent space colored by (left) predicted KMeans cluster assignments and (right) true genre labels.

latent becomes more structured when multiple information channels (audio reconstruction, lyric reconstruction, and conditional signal) shape the representation.

10.4 Metric limitations and dataset caveats

Internal metrics (silhouette, Davies–Bouldin, Calinski–Harabasz) are useful for model selection but can be misleading if cluster shapes are non-convex or if densities vary substantially. External metrics (ARI/NMI/purity) help interpret alignment with reference genre folders, but they depend on the quality of those labels and on dataset integrity.

GTZAN has been reported to include repetitions and labeling issues, which can influence both supervised and unsupervised evaluation. Therefore, results should be interpreted as performance on this edited GTZAN-derived subset rather than a definitive measure of real-world genre structure.

11 Reproducibility checklist (practical)

What is needed to rerun.

- A directory `genres_original/` containing one subdirectory per genre, each containing WAV files.
- (Medium/Hard) A matching `lyrics_generated/` directory with per-genre subfolders and per-track `.txt` files, or the code will treat missing lyrics as empty strings.
- Python packages: `librosa`, `scikit-learn`, `umap-learn`, `matplotlib`, `tqdm`, `torch`; and `sentence-transformers` for medium/hard.

Determinism notes. The easy task sets a fixed seed (42) for Python, NumPy, and PyTorch, which helps stabilize results. The medium/hard tasks fix random seeds in selected places (e.g., normalization subsampling and KMeans random state), but full determinism may still vary across GPU execution and library versions.

References

- [1] G. Tzanetakis and P. Cook. Musical genre classification of audio signals. *IEEE Transactions on Speech and Audio Processing*, 10(5):293–302, 2002.

- [2] Bob L. Sturm. The GTZAN dataset: Its contents, its faults, their effects on evaluation, and its future use. arXiv:1306.1461, 2013.
- [3] Bob L. Sturm. An analysis of the GTZAN music genre dataset. In *ACM MIRUM*, 2012.
- [4] Diederik P. Kingma and Max Welling. Auto-Encoding Variational Bayes. arXiv:1312.6114, 2013.
- [5] Irina Higgins et al. β -VAE: Learning basic visual concepts with a constrained variational framework. ICLR, 2017.
- [6] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. arXiv:1908.10084, 2019.
- [7] Leland McInnes, John Healy, and James Melville. UMAP: Uniform manifold approximation and projection for dimension reduction. arXiv:1802.03426, 2018.
- [8] Martin Ester, Hans-Peter Kriegel, Jörg Sander, and Xiaowei Xu. A density-based algorithm for discovering clusters in large spatial databases with noise. KDD, 1996.
- [9] S. P. Lloyd. Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2):129–137, 1982.